

Desenvolvimento e Análise de Modelos Clássicos de Recuperação de Informação

STUDENT

Ariel Alves da Silva (8847378)
Student 2 (student2_id) (8847378)

DATE

September 21, 2026

Contents

Introdução	2
Técnicas utilizadas	2
Tratamento de Documentos/Queries	2
Posting Lists	3
Modelo Probabilístico BM25	3
Modelo Vetorial	3
Avaliação dos modelos	4
Avaliação quantitativa	4

Introdução

Este trabalho consiste na implementação e análise de um sistema de Recuperação Textual utilizando dois modelos clássicos de recuperação da informação: modelo vetorial e modelo probabilístico. Ao analisar e comparar os dois modelos, é esperado obter uma distinção clara de classificação diante de uma base de dados real, fazendo uso de métricas exclusivas para avaliação de sistemas de Recuperação da Informação.

O sistema consiste basicamente em 4 etapas:

1. Processamento de documentos
2. Criação de posting lists
3. Processamento de query
4. Aplicação de query em modelos de RI

Todas foram implementadas na linguagem Python.

Técnicas utilizadas

Tratamento de Documentos/Queries

Para um processamento adequado dos documentos e queries, foi escolhido o uso da biblioteca NLTK (Natural Language Toolkit) do Python, que possui ferramentas úteis para realizar o processamento, entre elas:

- Lista de stopwords
- Lematização
- Stemming

As funções de tokenização e normalização foram aplicadas em hard-code.

Também foram criadas 4 possibilidades de pipeline:

- **NoStopRemovalNoStemming** - Sem stop words, sem stemming
- **NoStopRemovalWithStemming** - Sem stop words, com stemming
- **WithStopRemovalNoStemming** - Com stop words, sem stemming
- **WithStopRemovalWithStemming** - Com stop words, com stemming

Posting Lists

A posting list (ou índice invertido) é uma estrutura de dados que mapeia uma palavra específica para uma lista de documentos onde isto aparece, adicionado de informações opcionais.

Na implementação deste trabalho, a posting list é representada por uma classe que possui quatro mapeamentos:

1. **Postings** - Dicionário que mapeia termos para um dicionário de IDs que mapeia para a frequência do termo no documento
2. **Term Frequencies** - Dicionário que mapeia termos para a frequência total na posting list (`tf`)
3. **Document Frequencies** - Dicionário que mapeia termos para o número de documentos em que aparecem (`df`)
4. **Document Lengths** - Dicionário que mapeia IDs para o número total de termos no documento

As funções públicas da classe são para resgatar informações concernentes a estes quatro dicionários.

Modelo Probabilístico BM25

O modelo probabilístico BM25 consiste na seguinte função de ranqueamento:

$$\text{score}(d, q) = \sum_{t \in q} IDF(t) \frac{tf(t, d)(k_1 + 1)}{(tf(t, d) + k_1 \left(1 - b + \left(\frac{b(|d|)}{\text{avgdl}}\right)\right)}$$

- $tf(t, d)$ - Frequência do termo no documento
- k_1 - Parâmetro de controle da saturação
- b - Parâmetro de controle da influência do tamanho do documento
- $|d|$ - Tamanho do documento

Onde:

$$IDF(t) = \log \left(1 + \frac{N - n_t + 0.5}{n_t + 0.5} \right)$$

$$\text{avgdl} = \frac{1}{N} \sum_i^N |d_i|$$

- N = Quantidade total de documentos
- n_t = Quantidade de documentos que contém o termo t
- $|d_i|$ = Tamanho do documento

A implementação desta função foi feita exatamente como descrito aqui, com uso das bibliotecas `math` e `numpy`. Sua implementação foi feita como classe, recebendo os seguintes parâmetros:

- Posting List - Instância da classe `PostingList`
- Pipeline Options - Seleção de modo de processamento das queries

Modelo Vetorial

O modelo vetorial consiste na seguinte função de ranqueamento:

$$\text{sim}(d, q) = \frac{\sum w_{dt} w_{qt}}{\sqrt{\sum w_{dt}^2} + \sqrt{\sum w_{qt}^2}}$$

Onde:

$$w = \begin{cases} (1 + \log(f(t))) \cdot \log\left(\frac{N}{n_t}\right), & \text{se } f(t) > 0 \\ 0, & \text{se } f(t) = 0 \end{cases}$$

- $f(t)$ - Frequência do termo no documento/query.

Para otimizar o tempo de execução do algoritmo, o vetor de pesos é montado utilizando somente os termos pertencentes à query, com valor igual a zero caso algum dos termos seja inexistente no documento. Assim, se existem `n` termos na query, o vetor de qualquer documento selecionado terá dimensionalidade `n`, assim como o vetor de pesos da query.

A implementação desta função foi feita exatamente como descrito aqui, com uso das bibliotecas `math` e `numpy`. Sua implementação foi feita como classe, recebendo os seguintes parâmetros:

- Posting List - Instância da classe `PostingList`
- Pipeline Options - Seleção de modo de processamento das queries

Avaliação dos modelos

Ambos os modelos foram avaliados sob as seguintes condições:

- Para ambos, foram testados os 4 tipos de pipeline
- Para o modelo probabilístico, foram testadas as 9 combinações possíveis para os parâmetros k_1 e b

$$k_1 \in \{0.5, 1.2, 2.0\}$$

$$b \in \{0, 0.75, 1\}$$

As seguintes queries foram executadas:

1. Q1: what similarity laws must be obeyed when constructing aeroelastic models of aircraft
2. Q2: what are the aeroelastic problems associated with flight of high speed aircraft
3. Q3: what problems of transient heat conduction in composite slabs have been solved so far
4. Q4: what chemical kinetic mechanism is applicable to hypersonic aerodynamic problems
5. Q5: how can the aerodynamic performance of ground effect machines be calculated

Avaliação quantitativa

Para realizar a avaliação quantitativa entre modelos, foram selecionadas as seguintes métricas: Precision@10, Recall@10, MAP@10.